

Assignment No. 2
EECS 348
Software Engineering I
Antonio DePaolis
Due: 11:59 PM, Thursday, September 17, 2026

Analysis

1. AI used
 - a. CoPilot & Gemini
2. How I accessed AI
 - a. I accessed Copilot by going onto the copilot website. I accessed Gemini by going to the Google Gemini website
3. Prompt

Using C generate code that will take in a file name from the user which holds a list of commands. Use a MaxHeap to implement a priority queue. The program will prioritize emails for a busy company CEO. This priority queue will shift emails towards the front of the queue based on priority status. The MaxHeap must be implemented using list-based implementation then use it to handle all email prioritizing for the CEO. Create the functions from scratch DO NOT include pre-existing heap modules. Do not forget to include edge cases and comments on every line to describe what each line does. Here is the Test File format for the commands

EMAIL <sender category>, <subject line>, <date>

Emails in CEO's inbox should be placed in the queue based on sender category and date. Here is the priority. Boss – read first, Subordinate – read next, Peer – read next, ImportantPerson – read next, OtherPerson – read last

If more than one from a sender then the newest should be read first

EMAIL is followed by space. The rest of the fields are delimited.

Assume <sender category> is one of the five strings listed above.

Assume <subject line> is a string which may contain spaces, but not commas

Assume <date> is in the format: MM-DD-YYY

NEXT

Next email for the CEO to read. Display the information on the terminal in the following format:

Next email:

Sender:

Subject:

Date:

READ

CEO has read email and dealt with it

COUNT

Display current count of unread emails by saying how many how many are to be read

4. Generated code

a. CoPilot

```
#include <stdio.h>          // For input/output
#include <stdlib.h>         // For malloc, free, exit
#include <string.h>         // For string operations
#include <ctype.h>          // For character checks

// -----
// EMAIL PRIORITY DEFINITIONS
// -----

// Function assigns numeric priority based on sender category
int getCategoryPriority(char *category) {
    // Compare category strings and return priority values
    if (strcmp(category, "Boss") == 0) return 5;           // Highest
priority
    if (strcmp(category, "Subordinate") == 0) return 4;
    if (strcmp(category, "Peer") == 0) return 3;
    if (strcmp(category, "ImportantPerson") == 0) return 2;
    if (strcmp(category, "OtherPerson") == 0) return 1;    // Lowest
priority
    return 0;                                              // Edge
case: unknown category
}

// Converts MM-DD-YYYY into an integer YYYYMMDD for easy comparison
int convertDateToInt(char *date) {
    int month, day, year;
    sscanf(date, "%d-%d-%d", &month, &day, &year);        // Parse
date fields
    return year * 10000 + month * 100 + day;              // Convert
to comparable integer
}

// -----
// EMAIL STRUCTURE
// -----
```

[illegible]

```

// Heapify upward (bubble up)
void heapifyUp(MaxHeap *heap, int index) {
    while (index != 0) {
        // Continue until root
        int parent = (index - 1) / 2;
        // Compute parent index
        if (heap->arr[index].priority > heap->arr[parent].priority) {
            swap(&heap->arr[index], &heap->arr[parent]);
            // Swap if child has higher priority
            index = parent;
            // Move upward
        } else {
            break;
            // Stop if heap property satisfied
        }
    }
}

// Heapify downward (bubble down)
void heapifyDown(MaxHeap *heap, int index) {
    while (1) {
        int left = 2 * index + 1;
        // Left child index
        int right = 2 * index + 2;
        // Right child index
        int largest = index;
        // Assume current is largest

        if (left < heap->size && heap->arr[left].priority > heap->arr[largest].priority)
            largest = left;
        // Left child larger

        if (right < heap->size && heap->arr[right].priority > heap->arr[largest].priority)
            largest = right;
        // Right child larger

        if (largest != index) {

```

```

        swap(&heap->arr[index], &heap->arr[largest]); // Swap
with larger child
        index = largest; //
Continue downward
    } else {
        break; // Heap
property satisfied
    }
}

// Inserts an email into the heap
void insertEmail(MaxHeap *heap, Email email) {
    if (heap->size == heap->capacity) { // Edge
case: heap full
        printf("Heap is full, cannot insert email.\n");
        return;
    }

    heap->arr[heap->size] = email; // Insert
at end
    heapifyUp(heap, heap->size); // Restore
heap property
    heap->size++; //
Increase size
}

// Removes and returns highest priority email
Email extractMax(MaxHeap *heap) {
    if (heap->size == 0) { // Edge
case: empty heap
        Email empty = {"", "", "", 0};
        return empty;
    }

    Email maxEmail = heap->arr[0]; // Root is
highest priority
    heap->arr[0] = heap->arr[heap->size - 1]; // Move
last to root

```

```

        heap->size--;                                // Reduce
size
        heapifyDown(heap, 0);                        // Restore
heap property

        return maxEmail;                            // Return
removed email
    }

// -----
// COMMAND PROCESSING
// -----

// Parses EMAIL command and inserts into heap
void processEmailCommand(MaxHeap *heap, char *line) {
    char category[50], subject[200], date[20];

    // Extract fields after "EMAIL "
    sscanf(line + 6, "%[^,], %[^,], %s", category, subject, date);

    Email email;
    strcpy(email.sender, category);
    strcpy(email.subject, subject);
    strcpy(email.date, date);

    // Compute priority score
    int catPriority = getCategoryPriority(category);
    int datePriority = convertDateToInt(date);
    email.priority = catPriority * 100000000 + datePriority; // Combine
category + date

    insertEmail(heap, email);                        // Insert
into heap
}

// -----
// MAIN PROGRAM
// -----

```

```

int main() {
    char filename[100];
    printf("Enter command file name: ");
    scanf("%s", filename); // Read
file name from user

    FILE *fp = fopen(filename, "r"); // Open
file
    if (!fp) { // Edge
case: file not found
        printf("Error: Could not open file.\n");
        return 1;
    }

    MaxHeap *heap = createHeap(1000); // Create
heap with large capacity

    char line[300];
    while (fgets(line, sizeof(line), fp)) { // Read
each line
        if (strncmp(line, "EMAIL", 5) == 0) { // EMAIL
command
            processEmailCommand(heap, line);
        }
        else if (strncmp(line, "NEXT", 4) == 0) { // NEXT
command
            if (heap->size == 0) {
                printf("No emails to read.\n");
            } else {
                Email next = heap->arr[0]; // Peek
highest priority
                printf("Next email:\n");
                printf("Sender: %s\n", next.sender);
                printf("Subject: %s\n", next.subject);
                printf("Date: %s\n", next.date);
            }
        }
        else if (strncmp(line, "READ", 4) == 0) { // READ
command

```


b. Gemini

```
#include <stdio.h>    /* Standard input/output library for file
operations and printing */
#include <stdlib.h>    /* Standard library for dynamic memory allocation
(malloc, realloc, free) */
#include <string.h>    /* String library for string copying, comparison,
and manipulation */

/* Structure representing an Email record */
typedef struct {
    char sender[50];    /* Stores the category of the sender (e.g.,
"Boss", "Subordinate") */
    char subject[100]; /* Stores the subject line of the email */
    char date[11];      /* Stores the email date string in format
"MM-DD-YYYY" */
    int priority_score; /* Numeric score computed based on sender
category (higher = higher priority) */
    int date_score;     /* Numeric score computed based on date for
secondary comparison */
} Email;

/* Structure representing a dynamic, list-based MaxHeap priority queue
*/
typedef struct {
    Email *array;    /* Dynamic array storing the email elements of the
heap */
    int capacity;    /* Current maximum allocation capacity of the heap
array */
    int size;        /* Current number of elements stored in the heap */
} MaxHeap;

/* Forward declarations of functions */
MaxHeap* create_heap(int initial_capacity);
int get_sender_priority(const char *sender);
int parse_date_to_score(const char *date_str);
int compare_emails(Email e1, Email e2);
void swap_emails(Email *a, Email *b);
void heapify_up(MaxHeap *heap, int index);
```

```

void heapify_down(MaxHeap *heap, int index);
void insert_email(MaxHeap *heap, Email email);
Email peek_email(MaxHeap *heap, int *success);
Email extract_max(MaxHeap *heap, int *success);
void free_heap(MaxHeap *heap);
void trim_whitespace(char *str);

/* Main entry point of the program */
int main() {
    char filename[100];                /* Buffer to hold the input
file name from user */
    FILE *file;                       /* File pointer to handle
reading the command file */
    char line[256];                   /* Buffer to read line by line
from the input file */
    MaxHeap *heap = create_heap(10);  /* Initialize heap queue with an
initial capacity of 10 */

    /* Prompt user to enter the command file name */
    printf("Enter the command file name: ");
    if (scanf("%99s", filename) != 1) { /* Read file name safely from
standard input */
        printf("Error reading filename input.\n"); /* Handle input
reading failure */
        free_heap(heap);                /* Free memory before exit */
        return 1;                      /* Return error code 1 */
    }

    file = fopen(filename, "r");        /* Open the specified file in
read mode */
    if (file == NULL) {                /* Check if the file failed to
open (edge case: missing file) */
        printf("Error: Could not open file %s\n", filename); /* Display
error message */
        free_heap(heap);                /* Free allocated memory */
        return 1;                      /* Return error code 1 */
    }

    /* Loop to read command lines from the file line-by-line */

```

```

    while (fgets(line, sizeof(line), file) != NULL) {
        trim_whitespace(line);          /* Remove trailing newlines or
whitespace from line */
        if (strlen(line) == 0) continue; /* Skip empty lines (edge case:
blank lines in file) */

        /* Check if line starts with the "EMAIL" command */
        if (strncmp(line, "EMAIL ", 6) == 0) {
            char *data = line + 6;      /* Shift pointer past "EMAIL "
prefix */

            char *sender, *subject, *date; /* Pointers for
comma-delimited tokens */

            sender = strtok(data, ","); /* Tokenize string up to first
comma for sender */
            subject = strtok(NULL, ","); /* Tokenize string up to second
comma for subject */
            date = strtok(NULL, ",");   /* Tokenize string up to
end/comma for date */

            /* Validate that all three required fields exist (edge case:
malformed input) */
            if (sender && subject && date) {
                Email new_email;        /* Declare a temporary Email
struct */

                trim_whitespace(sender); /* Clean up extra space around
sender token */
                trim_whitespace(subject); /* Clean up extra space around
subject token */
                trim_whitespace(date);   /* Clean up extra space around
date token */

                /* Copy parsed values into the email structure fields
safely */
                strncpy(new_email.sender, sender,
sizeof(new_email.sender) - 1);
                new_email.sender[sizeof(new_email.sender) - 1] = '\0';
                /* Ensure null-termination */

```

```

        strncpy(new_email.subject, subject,
sizeof(new_email.subject) - 1);
        new_email.subject[sizeof(new_email.subject) - 1] = '\0';
/* Ensure null-termination */

        strncpy(new_email.date, date, sizeof(new_email.date) -
1);
        new_email.date[sizeof(new_email.date) - 1] = '\0'; /*
Ensure null-termination */

        /* Calculate priority score based on sender string */
        new_email.priority_score =
get_sender_priority(new_email.sender);
        /* Calculate chronological date score based on date
string */
        new_email.date_score =
parse_date_to_score(new_email.date);

        insert_email(heap, new_email); /* Insert the constructed
email into the priority heap */
    }
}
/* Check if line is the "NEXT" command */
else if (strcmp(line, "NEXT") == 0) {
    int success = 0; /* Flag to check if peek
operation succeeded */
    Email top = peek_email(heap, &success); /* Fetch highest
priority email without removing */
    if (success) { /* If heap was not empty */
        printf("Next email:\n"); /* Output formatted header */
        printf("Sender: %s\n", top.sender); /* Print sender
category */
        printf("Subject: %s\n", top.subject); /* Print subject
line */
        printf("Date: %s\n", top.date); /* Print date */
    } else { /* Handle empty heap edge case
*/
        printf("No emails in inbox.\n");
    }
}

```

```

    }
    /* Check if line is the "READ" command */
    else if (strcmp(line, "READ") == 0) {
        int success = 0; /* Flag to check if extraction
succeeded */
        extract_max(heap, &success); /* Remove highest priority
email (CEO dealt with it) */
        if (!success) { /* Handle edge case where heap
is already empty */
            printf("No emails to read.\n");
        }
    }
    /* Check if line is the "COUNT" command */
    else if (strcmp(line, "COUNT") == 0) {
        /* Display current unread count based on current heap size
*/
        printf("There are %d emails to be read.\n", heap->size);
    }
}

fclose(file); /* Close the input command file
safely */
free_heap(heap); /* Free all dynamically
allocated memory for the heap */
return 0; /* Program completed
successfully */
}

/* Creates and initializes a dynamic list-based MaxHeap */
MaxHeap* create_heap(int initial_capacity) {
    MaxHeap *heap = (MaxHeap*)malloc(sizeof(MaxHeap)); /* Allocate
memory for heap controller structure */
    if (!heap) { /* Handle memory allocation
failure edge case */
        printf("Memory allocation failure!\n");
        exit(1);
    }
    heap->capacity = initial_capacity; /* Set initial capacity */
    heap->size = 0; /* Set initial element count to

```

```

zero */
    heap->array = (Email*)malloc(sizeof(Email) * heap->capacity); /*
Allocate dynamic list array */
    if (!heap->array) { /* Handle memory allocation
failure edge case */
        printf("Memory allocation failure!\n");
        free(heap);
        exit(1);
    }
    return heap; /* Return pointer to
initialized heap */
}

/* Converts sender string category to a numerical priority (higher
integer = higher priority) */
int get_sender_priority(const char *sender) {
    if (strcmp(sender, "Boss") == 0) return 5; /* Top
priority */
    if (strcmp(sender, "Subordinate") == 0) return 4; /* 2nd
priority */
    if (strcmp(sender, "Peer") == 0) return 3; /* 3rd
priority */
    if (strcmp(sender, "ImportantPerson") == 0) return 2; /* 4th
priority */
    if (strcmp(sender, "OtherPerson") == 0) return 1; /* Lowest
priority */
    return 0; /* Fallback
for unknown category */
}

/* Parses date format "MM-DD-YYYY" into a comparable YYYYMMDD integer
score */
int parse_date_to_score(const char *date_str) {
    int month = 0, day = 0, year = 0; /* Local
integer storage for parsed date components */
    if (sscanf(date_str, "%d-%d-%d", &month, &day, &year) == 3) { /*
Parse formatted integers from string */
        return (year * 10000) + (month * 100) + day; /* Combine
into YYYYMMDD sortable representation */
    }
}

```

```

    }
    return 0; /* Return 0 on
failure to parse */
}

/* Compares two email items: returns >0 if e1 has higher priority than
e2, <0 if lower, 0 if equal */
int compare_emails(Email e1, Email e2) {
    if (e1.priority_score != e2.priority_score) { /* Check if
primary sender priorities differ */
        return e1.priority_score - e2.priority_score; /* Positive if
e1 has higher sender rank */
    }
    return e1.date_score - e2.date_score; /* Primary
ties broken by recency (newest/larger score wins) */
}

/* Swaps two Email elements in memory */
void swap_emails(Email *a, Email *b) {
    Email temp = *a; /* Store
element 'a' in temporary storage */
    *a = *b; /* Overwrite
element 'a' with element 'b' */
    *b = temp; /* Overwrite
element 'b' with temporary storage */
}

/* Restores max-heap property upwards starting from given index */
void heapify_up(MaxHeap *heap, int index) {
    while (index > 0) { /* Loop until
node reaches root */
        int parent = (index - 1) / 2; /* Compute
parent index in array representation */
        /* If current node has higher priority than its parent */
        if (compare_emails(heap->array[index], heap->array[parent]) > 0)
        {
            swap_emails(&heap->array[index], &heap->array[parent]); /*
Swap current node with parent */
            index = parent; /* Move index

```

```

up to parent position */
    } else {
        break; /* Heap
structure is valid, exit loop */
    }
}

/* Restores max-heap property downwards starting from given index */
void heapify_down(MaxHeap *heap, int index) {
    int max_index = index; /* Initialize
max index as current index */
    while (1) {
        int left = 2 * index + 1; /* Calculate
left child index */
        int right = 2 * index + 2; /* Calculate
right child index */

        /* Check if left child exists and has higher priority than
current max */
        if (left < heap->size && compare_emails(heap->array[left],
heap->array[max_index]) > 0) {
            max_index = left;
        }

        /* Check if right child exists and has higher priority than
current max */
        if (right < heap->size && compare_emails(heap->array[right],
heap->array[max_index]) > 0) {
            max_index = right;
        }

        /* If child was larger than current node, swap and keep
heapifying down */
        if (index != max_index) {
            swap_emails(&heap->array[index], &heap->array[max_index]);
            index = max_index; /* Update
current index to point to swapped child */
        } else {

```

```

        break; /* Heap
structure is valid, exit loop */
    }
}

/* Inserts a new email into the priority queue (MaxHeap) */
void insert_email(MaxHeap *heap, Email email) {
    /* Handle dynamic array resizing if capacity is reached (edge case:
list-based array expansion) */
    if (heap->size >= heap->capacity) {
        heap->capacity *= 2; /* Double
capacity */
        heap->array = (Email*)realloc(heap->array, sizeof(Email) *
heap->capacity); /* Reallocate memory */
        if (!heap->array) { /* Check for
allocation failure */
            printf("Memory allocation failure on expansion!\n");
            exit(1);
        }
    }
    heap->array[heap->size] = email; /* Place new
item at bottom end of heap */
    heap->size++; /* Increment
element count */
    heapify_up(heap, heap->size - 1); /* Bubbled up
to restore heap order */
}

/* Returns highest priority email without removing it */
Email peek_email(MaxHeap *heap, int *success) {
    if (heap->size == 0) { /* Edge case:
empty heap peek */
        *success = 0; /* Set failure
flag */
        Email empty = {"", "", "", 0, 0}; /* Return
dummy empty structure */
        return empty;
    }
}

```

```

        *success = 1;                                /* Set success
flag */
        return heap->array[0];                        /* Return root
element */
    }

/* Removes and returns highest priority email from the priority queue */
Email extract_max(MaxHeap *heap, int *success) {
    if (heap->size == 0) {                            /* Edge case:
empty heap extraction */
        *success = 0;                                /* Set failure
flag */
        Email empty = {"", "", "", 0, 0};            /* Return
dummy empty structure */
        return empty;
    }
    Email max_item = heap->array[0];                  /* Store max
element at root */
    heap->array[0] = heap->array[heap->size - 1];      /* Move last
element to root position */
    heap->size--;                                     /* Decrement
heap size */
    heapify_down(heap, 0);                           /* Bubble down
root to restore heap property */
    *success = 1;                                    /* Set success
flag */
    return max_item;                                  /* Return
extracted priority item */
}

/* Frees all dynamic memory associated with the heap queue */
void free_heap(MaxHeap *heap) {
    if (heap) {                                       /* Check if
heap pointer is non-null */
        if (heap->array) {                           /* Check if
internal dynamic list exists */
            free(heap->array);                         /* Free
dynamic memory array */
        }
    }
}

```

```

        free(heap); /* Free heap
container structure */
    }
}

/* Utility function to trim leading and trailing spaces/newlines from
strings */
void trim_whitespace(char *str) {
    char *end; /* Pointer
tracking end of string */
    while (*str == ' ' || *str == '\t' || *str == '\r' || *str == '\n')
str++; /* Skip leading whitespace */
    if (*str == 0) return; /* Handle
empty string */
    end = str + strlen(str) - 1; /* Point to
last character */
    while (end > str && (*end == ' ' || *end == '\t' || *end == '\r' ||
*end == '\n')) end--; /* Trim trailing */
    end[1] = '\0'; /* Write
null-terminator at end of trimmed string */
}

```

5. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.
 - a. Correctness

Both ran successfully with normal inputs

Testing Edgcases

- 1) CoPilot
 - a) Is able to NEXT or READ even if there are no emails left and prints “No emails to read.”
 - b) Ignores unknown commands
 - c) Fails to handle EMAIL with missing parameters
- 2) Gemini
 - a) Is able to NEXT or READ even if there are no emails left and prints “No emails to read.” for READ and “No emails in inbox.” for NEXT
 - b) Ignores unknown commands
 - c) If EMAIL doesn’t have all parameters it handles it by ignoring it

- b. Execution time

- 1) Time Complexity
 - a) Both Are the same as they both use a binary MaxHeap $O(m\log(m))$
 - i) Use a while loop to read the file
 - ii) Use a MaxHeap to hold a priority list with the insert
- 2) Execution Time
 - a) Both
 - i) Both use a while loop to read through the file
 - ii) The commands are all similar in execution with no big differences asides from one having more checks than the other
 - b) CoPilot
 - i) Has less checks which result in less time taken during each line command
 - c) Gemini
 - i) Has more checks done for each line command which can add up

- c. Space Complexity

- 1) Both
 - a) They both use $O(n)$ space as they both store the number of emails the file has into a heap everything else is just temporary constants

d. Maintainability

1. Both
 - a. Variable naming for both match the structure
2. CoPilot
 - a. Code is shorter and simpler
 - b. There are less functions
 - c. Comments aren't as descriptive as they should be
 - d. Missing some edge cases such as empty EMAIL commands or just whitespace
3. Gemini
 - a. Code is more modular with multiple functions to perform different parts like getting the sender priority. Or just to perform string modification like trimming whitespace.
 - b. Comments are more descriptive on each line
 - c. Handles many edge cases if not all
 - d. Error handling is fine but could be expanded

6. Select Which AI and explain why
 - a. I decided to choose the Gemini Code as it has most of the edge cases handled. It is easier to read and has described its code better. It is also more modular with many functions performing different actions to be utilized by the main. CoPilot code seems to be more simple with a lot of the code being done within the main function rather than in functions.
7. Explain how you would improve the code that you chose
 - a. To improve the code I would improve maintainability by adding more comments on lines that didn't receive comments and changing them. I will improve the trim-whitespace function to properly handle leading whitespace. Time complexity and space complexity cannot really be significantly modified.